

TCP Guide: Wireshark Analysis of TCP Traffic

What is TCP?

TCP (Transmission Control Protocol) is the reliable, connection-oriented transport layer protocol that powers most internet communication. It ensures data is delivered accurately, in order, and without loss — forming the foundation for higher-level protocols like HTTP, HTTPS, FTP, SMTP, etc.

TCP is where you see the actual **handshakes**, **data flow**, **acknowledgments**, and **connection management**.

Typical Ports

TCP uses **ephemeral ports** (usually 1024–65535) on the client side and well-known ports on the server side:

- 80 (HTTP), 443 (HTTPS)
- 21 (FTP control), 22 (SSH)
- 25 (SMTP)
- Many others

In analysis, you focus on the **source and destination ports** to understand which service is being used.

Why Analysts Care About TCP

- Almost all application-layer protocols (HTTP, DNS over TCP, etc.) ride on top of TCP.
- Many attacks target the TCP layer: SYN floods, session hijacking, port scanning, reset attacks, etc.
- Understanding TCP lets you reconstruct conversations, detect data loss or retransmissions, and analyze connection behavior.
- You can spot anomalies such as unusual flags, fragmentation, or suspicious port knocking.

Wireshark Fundamentals for TCP Analysis

1. Basic Filtering

Bash

```
tcp                # All TCP traffic
tcp.port == 80     # Traffic involving port 80
tcp.flags.syn == 1 # SYN packets (connection start)
tcp.flags.ack == 1 # ACK packets
tcp.flags.rst == 1 # RST (reset) packets
tcp.flags.fin == 1 # FIN (graceful close)
tcp.analysis.retransmission # Packet retransmissions
tcp.stream eq 0    # Specific TCP stream (conversation)
```

2. Following Conversations

- Right-click any TCP packet → **Follow** → **TCP Stream**
- This shows the reconstructed bidirectional data (very useful even for non-HTTP protocols).

3. Key TCP Fields to Master

TCP Header Fields

- **Source Port / Destination Port** — Identifies client and service.
- **Sequence Number (SEQ)** — Tracks order of bytes sent.
- **Acknowledgment Number (ACK)** — Confirms receipt of data.
- **Flags** (very important):
 - **SYN** — Synchronize (start connection)
 - **ACK** — Acknowledgment
 - **RST** — Reset (abrupt close)
 - **FIN** — Finish (graceful close)
 - **PSH** — Push (deliver data immediately)
 - **URG** — Urgent
- **Window Size** — Flow control.
- **Checksum** — Error detection.

Three-Way Handshake (The Most Important Pattern)

1. Client → Server: **SYN**
2. Server → Client: **SYN + ACK**
3. Client → Server: **ACK**

This establishes a reliable connection.

Connection Termination

- Graceful: FIN → ACK → FIN → ACK (four-way)
- Abrupt: RST

Common TCP Analysis Scenarios in Thaghrach Challenges

1. **Three-Way Handshake Analysis**
 - Verify if connections are established normally.
 - Spot half-open connections or SYN floods.
2. **Port Scanning / Port Knocking**
 - Look for rapid SYN packets to many ports (scanning).
 - Port knocking: specific sequence of closed-port SYNs to open a hidden service.
3. **Data Fragmentation & Reassembly**
 - Large files or responses split across many TCP segments.
 - Wireshark automatically reassembles streams when you Follow TCP Stream.
4. **Retransmissions & Performance Issues**

- tcp.analysis.retransmission — indicates packet loss or congestion.
- Duplicate ACKs and Zero Window conditions.
- 5. **Session Hijacking / Reset Attacks**
 - Unexpected RST packets in the middle of a conversation.
 - Sequence number prediction (rare but educational).
- 6. **Mixed Traffic Analysis**
 - Multiple protocols (e.g., HTTP + DNS) in the same capture.

Practical Workflow in Wireshark for TCP

1. Open the PCAP and start with tcp filter.
2. Use the **TCP Stream** graph: **Statistics** → **TCP Stream Graphs** → **Time/Sequence/Throughput**.
3. Colorize rules (recommended):
 - SYN/FIN packets in one color.
 - RST packets in red (often suspicious).
4. Right-click → **Follow** → **TCP Stream** for readable data.
5. Use **Statistics** → **Conversations** → **TCP** to see top talkers and ports.
6. Export objects or raw streams when needed.

Security Red Flags in TCP Traffic

- Excessive SYN packets without completing handshakes (scanning/flooding).
- RST packets interrupting legitimate sessions.
- Unusual source ports or port patterns.
- Data sent before handshake completes (rare, suspicious).
- Very small or very large window sizes (possible evasion).
- Connections to unexpected high ports.

Example Challenge Goals You Will Encounter

- Reconstruct the full conversation from fragmented TCP segments.
- Identify which port knocking sequence opens a hidden service.
- Find the flag hidden in the TCP payload of a specific stream.
- Detect and explain an anomalous TCP behavior (e.g., connection reset or retransmission storm).
- Determine the exact bytes transferred during a file download.

Tips for Thaghras Students:

- Always start by identifying the **TCP streams** involved in the challenge.
- Use the handshake to confirm when a connection begins.
- Pay close attention to **sequence numbers** when data appears out of order.
- Combine TCP analysis with higher-layer filters (e.g., tcp && http).